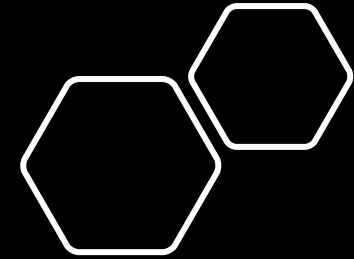


Java Data Types

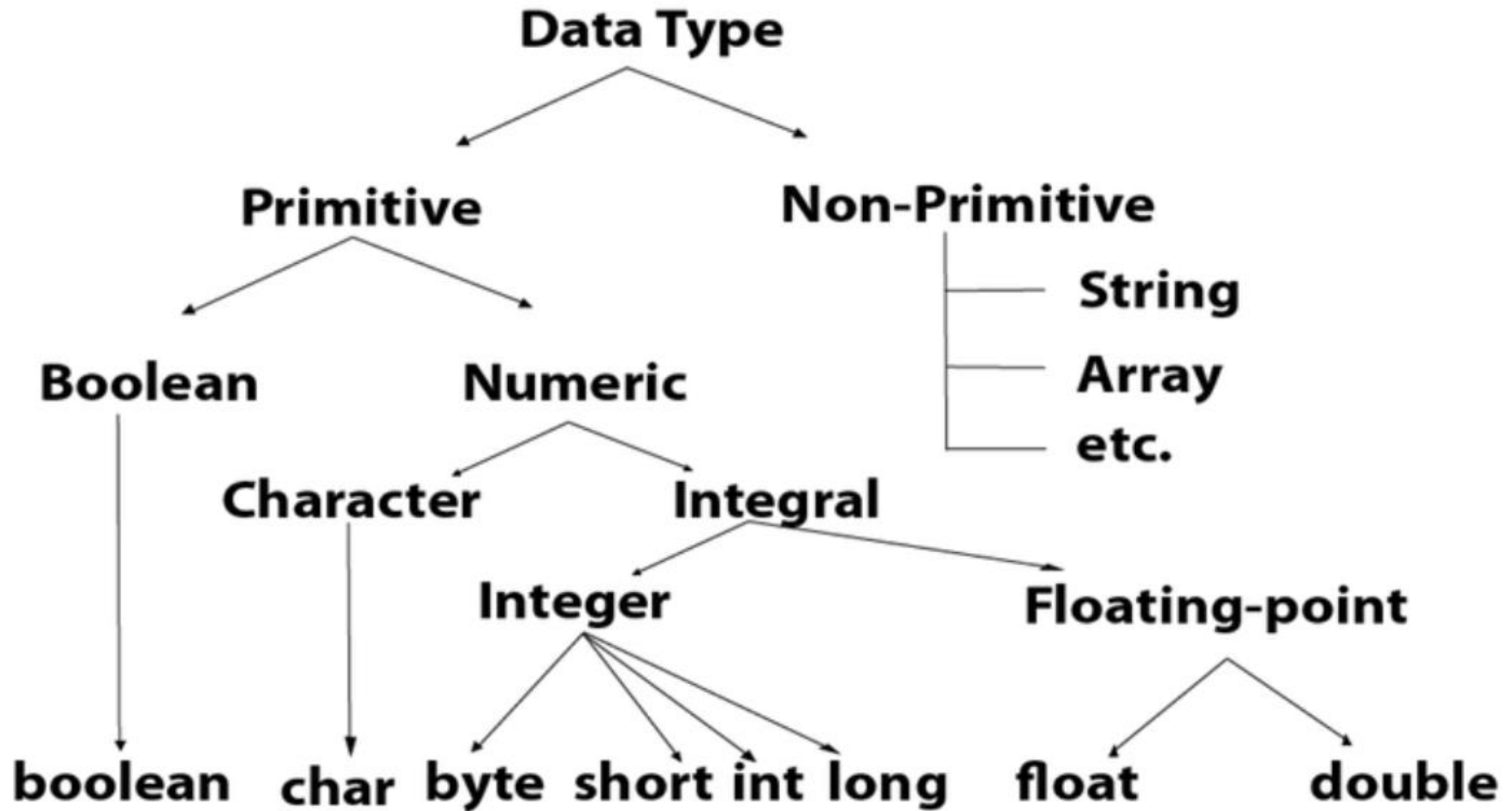


Java™

Data types specify the different sizes and values that can be stored in the variable. There are two types of data types in Java:

Primitive data types: The primitive data types include boolean, char, byte, short, int, long, float and double.

Non-primitive data types: The non-primitive data types include Classes, Interfaces, and Arrays.



INTEGER DATA TYPE

- The int data type is a 32-bit signed two's complement integer. Its value-range lies between - 2,147,483,648 (-2^{31}) to 2,147,483,647 ($2^{31} - 1$)
- The int data type is generally used as a default data type for integral values unless if there is no problem about memory.
- Example: `int a = 100000`, `int b = -200000`

SHORT DATA TYPE

- The short data type is a 16-bit signed two's complement integer. Its value-range lies between -32,768 to 32,767 (inclusive).
- Its minimum value is -32,768 and maximum value is 32,767. Its default value is 0.
- The short data type can also be used to save memory just like byte data type.
- A short data type is 2 times smaller than an integer.
- Example: short s = 10000, short r = -5000

BYTE DATA TYPE

- The byte data type is an example of primitive data type.
- It is an 8-bit signed two's complement integer. Its value-range lies between -128 to 127 (inclusive).
- Its minimum value is -128 and maximum value is 127. Its default value is 0.
- The byte data type is used to save memory in large arrays where the memory savings is most required.
- It saves space because a byte is 4 times smaller than an integer. It can also be used in place of "int" data type.
- Example: byte a = 10, byte b = -20

LONG DATA TYPE

- The long data type is a 64-bit two's complement integer.
- The long data type is used when you need a range of values more than those provided by int.
- Example: `long a = 100000L`, `long b = -200000L`

FLOAT DATA TYPE

- The float data type is a single-precision 32-bit IEEE 754 floating point.
- Its value range is unlimited.
- It is recommended to use a float (instead of double) if you need to save memory in large arrays of floating point numbers.
- The float data type should never be used for precise values, such as currency. Its default value is 0.0F.
- Example: `float f1 = 234.5f`

DOUBLE DATA TYPE

- The double data type is a double-precision 64-bit IEEE 754 floating point.
- Its value range is unlimited.
- The double data type is generally used for decimal values just like float.
- The double data type also should never be used for precise values, such as currency.
- Its default value is 0.0d.
- Example: double d1 = 12.3

CHAR DATA TYPE

char: The char data type is a single 16-bit Unicode character. It has a minimum value of '\u0000' (or 0) and a maximum value of '\uffff' (or 65,535 inclusive).

INPUT AND OUTPUT

□ IO STREAMS

□ **System.in**

□ **System.out**

□ **System.err**

INPUT AND OUTPUT

OUTPUT

System.out.print()

System.out.println()

System.out.printf()

```
System.out.print("Hello,");  
System.out.print(" world!");
```

Hello, world!

```
System.out.println("Hello,");  
System.out.println("world!");
```

Hello,
world!

```
System.out.printf("Hello, %s. You have  
%d new messages.%n", "Alice", 5);
```

Hello, Alice. You have 5 new messages.

Keywords In Java

- | | | | |
|--------------|----------------|---------------|------------------|
| 1. abstract | 13. double | 25. int | 37. strictfp |
| 2. assert | 14. else | 26. interface | 38. super |
| 3. boolean | 15. enum | 27. long | 39. switch |
| 4. break | 16. extends | 28. native | 40. synchronized |
| 5. byte | 17. final | 29. new | 41. this |
| 6. case | 18. finally | 30. package | 42. throw |
| 7. catch | 19. float | 31. private | 43. throws |
| 8. char | 20. for | 32. protected | 44. transient |
| 9. class | 21. if | 33. public | 45. try |
| 10. continue | 22. implements | 34. return | 46. void |
| 11. default | 23. import | 35. short | 47. volatile |
| 12. do | 24. instanceof | 36. static | 48. while |

```
class Student
{
public static void main(String args[])
{
System.out.println("my first program");
}
}
```

The array of strings can be declared in different ways

public static void main(String args[])

public static void main(String[] args)

public static void main(String args[])

public: This is an access modifier. public means that the method can be called from outside of its current class. For the main method, this is required because it's the entry point of the program and must be accessible by the Java runtime environment.

static: This means that the method can be called without creating an instance of the class. This is necessary for the main method because it's called by the JVM before any objects are made.

void: This indicates that the method doesn't return any value.

This is the method that the JVM looks for as the starting point of the program.

(String args[]): The main method takes a single argument: an array of strings. This array contains any command-line arguments that were passed to the program. In this simple example, the command-line arguments are not used, but they could be accessed within the main method if needed

It should start with the uppercase letter.

CLASS

Same for multiple words

This keyword is used to declare a class in Java. A class is a blueprint from which individual objects are created. In Java, every program must have at least one class.

Ex : Student

Ex : BranchName

METHOD

In Java, methods are blocks of code that perform specific tasks and are defined within classes or interfaces.

First letter should begin with lower case

For multiple words, after first letter all must be in upper case letter

Ex : read() , getName()

Ex : readName(), nextLine()

VARIABLE

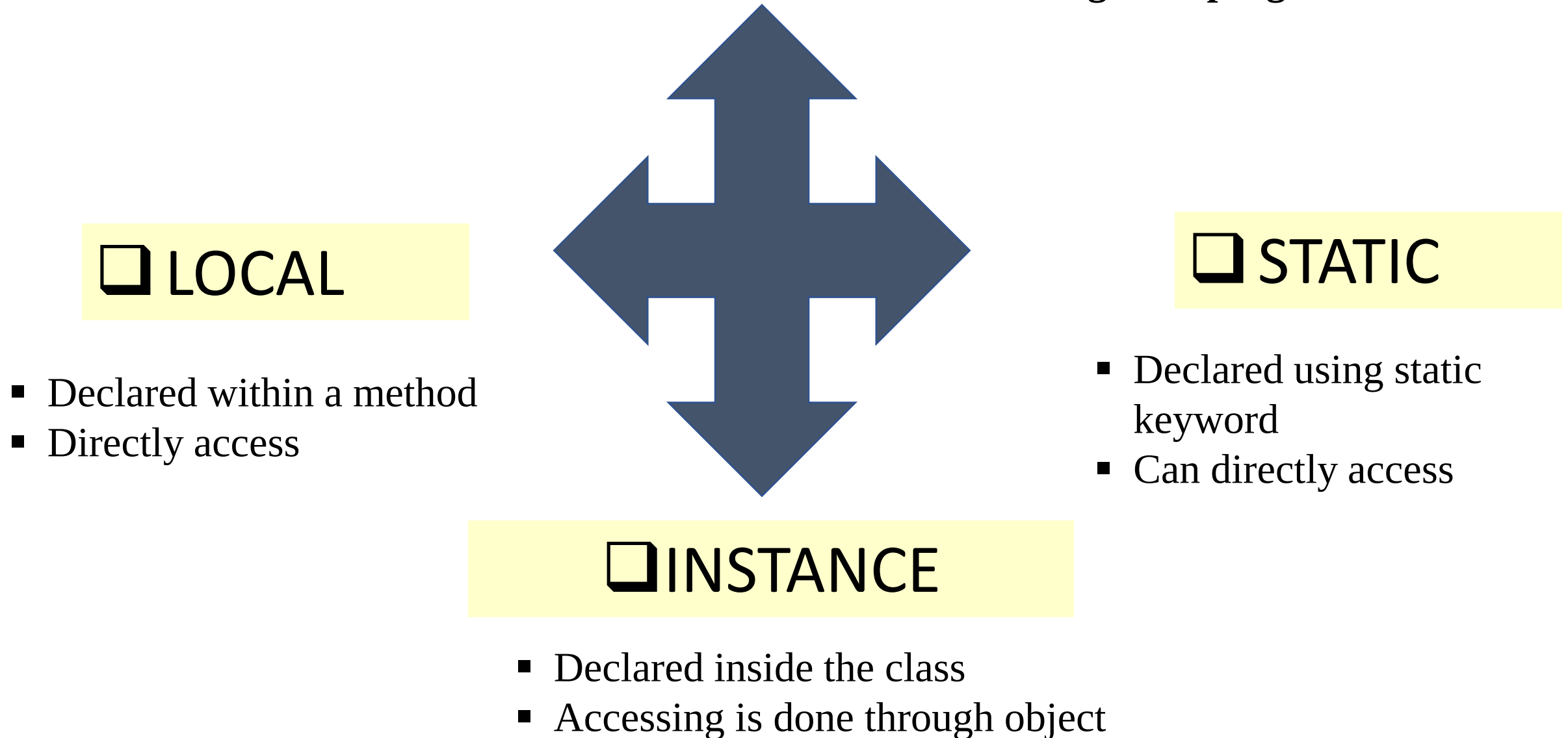
Begin with Alphabet or underscore
It should start with a lowercase letter such as id, name.

Shouldn't begin with
Digits and It should not start with the special characters like &
(ampersand), \$ (dollar).

```
class Employee
{
    int id;
}
```

VARIABLES

Variable in Java is a **data container** that saves the data values during Java program execution.



VARIABLES

```
class Variable
{
int a = 10; /*INSTANCE VARIABLE*/
public static void main(String args[])
{
    Variable v = new Variable();
    System.out.println(v.a);
}
}
```

VARIABLES

```
class Variable
{
static int c = 30; /*STATIC VARIABLE*/
int a = 10; /*INSTANCE VARIABLE*/
public static void main(String args[])
{
    int b = 20; /*LOCAL VARIABLE*/
    Variable v = new Variable();
    System.out.println(v.a);
    System.out.println(b);
    System.out.println(c);

}
}
```

Java Variable Example: Add Two Numbers

Local variable

```
1.  class Simple{  
2.  public static void main(String[] args){  
3.  int a=10;  
4.  int b=10;  
5.  int c=a+b;  
6.  System.out.println(c);  
7.  }}
```

Java Variable Example: FLOAT

```
1. class Simple{  
2. public static void main(String[] args){  
3. int a=10;  
4. float f=a;  
5. System.out.println(a);  
6. System.out.println(f);  
7. }}
```

1. Output: 10
10.0

SAVE – CLASSNAME.JAVA

TO COMPILE – JAVAC FILENAME.JAVA (FILE
NAME NEED TO BE THE CLASS NAME)

Set Path = C:\

Set HomePath= C:\Program Files\java\jdk